

先来张图感受一下MRoPE，其原理是将hidden_dim分为三个部分，分别用来编码Time、Height、Width三个维度：



虽然了解了他的原理，但是我们对其内部还是知之甚少，因此我们来仔细分析，[Qwen-VL](#) 的代码是如何实现的：

1. 计算 position_ids

首先需要计算一个三维IDs序列表示每一个token（视觉的+文本的）在时空中的位置，这里我们用一个例子就可以说明具体做的事情：

对于纯文本嵌入序列，旋转位置嵌入与现代大语言模型中的方法没有区别。例如：

- input_ids: [T T T T T]，其中 T 表示文本。
- temporal position_ids: [0, 1, 2, 3, 4]。
- height position_ids: [0, 1, 2, 3, 4]。
- width position_ids: [0, 1, 2, 3, 4]。

对于包含视觉和文本嵌入的序列，我们对视觉部分计算三维旋转位置嵌入（3D rotary position embedding），而对文本部分计算一维旋转位置嵌入（1D rotary position embedding）。例如：

假设我们有一个视频输入，其包含 3 个 temporal patch、2 个 height patch 和 2 个 width patch。

- input_ids: [V V V V V V V V V V V V T T T T T]，其中 V 表示视觉。
- 视觉部分：
 - temporal position_ids: [0, 0, 0, 0, 1, 1, 1, 1, 2, 2, 2, 2]。
 - height position_ids: [0, 0, 1, 1, 0, 0, 1, 1, 0, 0, 1, 1]。

- width position_ids: [0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1] 。
- 文本部分：
 - temporal position_ids: [3, 4, 5, 6, 7] 。
 - height position_ids: [3, 4, 5, 6, 7] 。
 - width position_ids: [3, 4, 5, 6, 7] 。

其中，文本部分的起始 position_ids 是由视觉部分的最大 position_ids 加 1 得到的。

具体的实现参考[源码](#)的 `Qwen2VLForConditionalGeneration.get_rope_index` 。

2. 根据 position_ids 计算 3D Embedding

首先先看一下MRoPE如何预计算的角度信息：首先计算 $\text{head_dim}/2$ 个 θ_i 角度信息，然后与 position_id 进行相乘得到 freqs，最后两个相同的 freqs 进行拼接得到 emb，第 ids 位置token的 freqs为 $\left[i \cdot d \cdot \theta_1, i \cdot d \cdot \theta_2, \dots, i \cdot d \cdot \theta_{d/2}, i \cdot d \cdot \theta_1, i \cdot d \cdot \theta_2, \dots, i \cdot d \cdot \theta_{d/2} \right]$ ：

```
class Qwen2VLRotaryEmbedding(nn.Module):
    def forward(self, x, position_ids):
        if "dynamic" in self.rope_type:
            self._dynamic_frequency_update(position_ids, device=x.device)

        # Core RoPE block. In contrast to other models, Qwen2_VL has
        different position ids for thw grids
        # So we expand the inv_freq to shape (3, ...)
        # 首先计算head_dim//2个\theta_i角度信息，并expand到[3, 1, 64, 1]，
        其中head_dim=128
        inv_freq_expanded = self.inv_freq[None, None, :, None].float().expand(3, position_ids.shape[1], -1, 1) # [3, 1, 64, 1]
        position_ids_expanded = position_ids[:, :, None, :].float()
        # shape (3, bs, 1, positions)
        # Force float32 (see https://github.com/huggingface/transformers/pull/29285)
        device_type = x.device.type
        device_type = device_type if isinstance(device_type, str) and
```

```

device_type != "mps" else "cpu"
    with torch.autocast(device_type=device_type, enabled=False):
        # \theta_i与position_ids进行相乘，得到ids * \theta_i，这是一个
        二维矩阵
        freqs = (inv_freq_expanded.float() @ position_ids_expanded.float()).transpose(2, 3) # [3, bs, positions, 64]
        # 相同的freqs进行拼接
        emb = torch.cat((freqs, freqs), dim=-1) # [3, bs, positions, 128]

        cos = emb.cos()
        sin = emb.sin()

        # Advanced RoPE types (e.g. yarn) apply a post-processing scaling factor, equivalent to scaling attention
        cos = cos * self.attention_scaling
        sin = sin * self.attention_scaling

        return cos.to(dtype=x.dtype), sin.to(dtype=x.dtype)

```

这里需要解释一下为什么hidden_dim是128，可以参考 [config.json](#)：

```

"hidden_size": 3584
"num_attention_heads": 28

```

因此里是 $\text{head_dim} = 3584 / 28 = 128$

3. 拼接 3D Embedding

上一步我们分别获得了所有位置的Time、Width、Height维度的Pos Embedding，其维度为[3, bs, positions, 128]。但是我们在实际使用位置编码时候只能使用[bs, positions, 128]维度的向量，因此涉及一个问题：**如何把三个维度的位置编码拼接（压缩）起来？**

答案是每个维度切取一部分的位置编码，然后进行拼接得到最终的旋转位置编码，这样就包含了不同维度的位置信息。公式和code实现如下：

$$R_{\Theta,ids_t,ids_x,ids_y}^d x = \begin{pmatrix} x_1 \\ \vdots \\ x_{16} \\ x_{17} \\ \vdots \\ x_{40} \\ x_{41} \\ \vdots \\ x_{64} \\ x_{65} \\ \vdots \\ x_{80} \\ x_{81} \\ \vdots \\ x_{104} \\ x_{105} \\ \vdots \\ x_{128} \end{pmatrix} \otimes \begin{pmatrix} \cos ids_t \theta_1 \\ \vdots \\ \cos ids_t \theta_{16} \\ \cos ids_x \theta_1 \\ \vdots \\ \cos ids_x \theta_{24} \\ \cos ids_y \theta_1 \\ \vdots \\ \cos ids_y \theta_{24} \\ \cos ids_t \theta_1 \\ \vdots \\ \cos ids_t \theta_{16} \\ \cos ids_x \theta_1 \\ \vdots \\ \cos ids_x \theta_{24} \\ \cos ids_y \theta_1 \\ \vdots \\ \cos ids_y \theta_{24} \end{pmatrix} + \begin{pmatrix} -x_{65} \\ \vdots \\ -x_{80} \\ -x_{81} \\ \vdots \\ -x_{104} \\ -x_{105} \\ \vdots \\ -x_{128} \\ x_1 \\ \vdots \\ x_{16} \\ x_{17} \\ \vdots \\ x_{40} \\ x_{41} \\ \vdots \\ x_{64} \end{pmatrix} \otimes \begin{pmatrix} \sin ids_t \theta_1 \\ \vdots \\ \sin ids_t \theta_{16} \\ \sin ids_x \theta_1 \\ \vdots \\ \sin ids_x \theta_{24} \\ \sin ids_y \theta_1 \\ \vdots \\ \sin ids_y \theta_{24} \\ \sin ids_t \theta_1 \\ \vdots \\ \sin ids_t \theta_{16} \\ \sin ids_x \theta_1 \\ \vdots \\ \sin ids_x \theta_{24} \\ \sin ids_y \theta_1 \\ \vdots \\ \sin ids_y \theta_{24} \end{pmatrix}$$

至于time、x、y三个维度是怎么划分hidden_dim的，是在<https://huggingface.co/Qwen/Qwen2-VL-7B-Instruct/blob/main/config.json>里的rope_scaling["mrope" section]参数，里面将64个角度划分成了[16, 24, 24]一组的三份，在实际切分方法 apply_multimodal_rotary_pos_emb 中，mrope_section进行一次复制成为[16, 24, 24, 16, 24, 24]，正好划分128个维度：

```
# position_ids是一个[3, b, num_tokens]，每个token有3个方向temporal, height and width的位置编码id
# cos.shape = [3,b,num tokens, 128]
# sin.shape = [3,b,num_tokens, 128]
# q,k.shape = [b,n_head,num_tokens,dim] dim=128
def apply_multimodal_rotary_pos_emb(q, k, cos, sin, mrope_section, unsqueeze_dim=1):
    # mrope section = [16, 24, 24, 16, 24, 24]
    mrope_section = mrope_section * 2
    # 先将cos对dim维度划分为6组数据，[3, 1, num_tokens, 16], [3, 1, num_tokens, 24], ...
    # (1) 第1组数据slice[0,...]即(temporal)出来[1,num_tokens,16]。
    # (2) 第2组数据slice[1,...]即(height)出来[1,num_tokens,24]
```

```

# (3) 第3组数据slice[2,...]即(width)出来[1,num_tokens,24]
# (4) 第4组数据slice[0,...]即(temporal)出来[1,num_tokens,16],由于freqs(128)前半(64)和后半(64)相同,这一块的freqs则与(1)中的freqs是相同的
# (5) 第5组数据slice[1,...]即(height)出来[1,num_tokens,24]
# (6) 第6组数据slice[2,...]即(width)出来[1,num_tokens,24]
# cat拼接为[1,1,num_tokens,128]
cos = torch.cat([m[i % 3] for i, m in enumerate(cos.split(mrope_section, dim=-1))], dim=-1).unsqueeze(
    unsqueeze_dim
)
# sin处理和cos一样的
sin = torch.cat([m[i % 3] for i, m in enumerate(sin.split(mrope_section, dim=-1))], dim=-1).unsqueeze(
    unsqueeze_dim
)
# 0-15特征元素使用temporal类型的位置编码, 16-39使用height类型的位置编码,
...
q_embed = (q * cos) + (rotate_half(q) * sin)
k_embed = (k * cos) + (rotate_half(k) * sin)
return q_embed, k_embed

```

参考阅读：<https://zhuanlan.zhihu.com/p/9613363595> (2D-RoPE的推导不正确，MRoPE可以看)